SEEK-O-SPHERE

Seek-O-Sphere

Project Report

A free, interactive website that teaches Artificial Intelligence to everyone, ages 8 to 80 — built with plain HTML, CSS and JavaScript on a Firebase backend.

Project: Seek-O-Sphere · **Developer:** Soumil · **Stack:** HTML/CSS/JS + Firebase · **Hosting:** GitHub Pages

Contents

1. Abstract

Seek-O-Sphere is a free, browser-based educational platform designed to make Artificial Intelligence understandable and approachable for learners of every age, from curious eight-year-olds to lifelong learners of eighty. It combines plain-language chapters, an interactive history of AI, a hands-on practice Lab with curriculum-matched and competitive-exam questions, an AI-tool recommender, and a personal progress dashboard. The application is built entirely with vanilla HTML, CSS and JavaScript — no frameworks or build tools — and uses Google Firebase (Authentication and Cloud Firestore) for accounts and progress tracking. The result is a lightweight, fast, and maintainable learning site that runs anywhere a modern browser does and is hosted for free on GitHub Pages.

2. Introduction

Artificial Intelligence has moved from research labs into everyday life, yet clear, friendly, age-appropriate learning material remains scarce. Most explanations are either too technical for beginners or too shallow to be useful. Seek-O-Sphere addresses this gap with a single, cohesive experience that meets learners where they are: it explains what AI is and how it evolved, lets learners practise real questions matched to their school curriculum or exam goals, and helps them choose the right AI tool for a task — all wrapped in a warm, playful interface guided by a friendly mascot named **Eulid**.

The project deliberately avoids heavy tooling. By using only the native web platform, it stays small, loads quickly, is easy to host, and is straightforward to maintain and extend — an intentional design choice that also makes the codebase an excellent teaching artifact in its own right.

3. Objectives

- Teach AI concepts in plain language to a very wide age range (8–80) without jargon.
- Provide active, hands-on practice through a Lab that adapts to a learner's curriculum, age group and subject.
- Offer competitive-exam preparation (CUET, Railways, Bank PO, SSC, UPSC) alongside school practice.
- Track each learner's progress — accuracy, streaks and activity — and present it clearly on a personal dashboard.
- Give educators/administrators visibility into all learners through a secure admin panel.
- Keep the platform free, fast, and dependency-free, hostable on static infrastructure.

4. Scope of the Project

The platform covers the full learner journey: discovering and reading about AI, practising questions, tracking personal progress, and (for staff) monitoring cohort activity. In scope are the public learning experience, account creation and authentication, per-learner progress persistence, the student dashboard, and the admin panel. Out of scope (by design) are a server-side application tier, native mobile apps, and automated content authoring — the site is a static front end backed by Firebase's managed services.

5. Technology Stack

Layer	Technology	Why
Markup	HTML5 — single-page app + auth pages	Native, zero-build, universally supported
Styling	Vanilla CSS, custom "Sienna" design system	Full control, no framework weight
Scripting	Vanilla JavaScript (ES6+)	No build step; runs directly in the browser
Fonts	Google Fonts — Nunito, Inter	Friendly headings, readable body
Questions	Local engine (algorithmic + JSON), optional Claude API	Free offline mode with an AI upgrade path
Authentication	Firebase Auth (compat v10.12.0)	Email, Google and guest sign-in out of the box
Database	Cloud Firestore (compat v10.12.0)	Real-time, offline-capable document store
Hosting	GitHub Pages	Free static hosting with Git-based deploys

6. System Architecture

Seek-O-Sphere is a static front end talking directly to Firebase's managed backend — there is no custom server. It has two complementary front-end surfaces:

The learning app — `index.html`

A self-contained single-page application. Every learning section (Home, What is AI?, History, Types, Study Tools, Ethics, Lab, Find My AI Tool, Resources, Help, Contact) is a `.panel` shown and hidden by a `showPage()` router. All of its CSS and JavaScript are inline for portability.

The account flows — `pages/`

Standalone pages for authentication and personal data: `login.html`, `dashboard.html`, `admin.html` and `account-settings.html`, plus a set of standalone content pages that share `js/core/nav.js`.

A shared session layer (`js/session.js`) initialises Firebase, renders the account menu, and reconciles a fast `localStorage` cache with the live user profile so names and avatars appear instantly. Firestore's offline persistence is enabled so reads come from a local cache and writes commit locally first, then sync — which keeps the interface responsive and functional even on flaky connections.

```
Browser (static files: HTML/CSS/JS)
   │
   ├─ Firebase Authentication ──▶ identity (email / Google / guest)
   │
   └─ Cloud Firestore ─────────▶ users/{uid}  +  progress/{uid}/daily/*
        (offline persistence: local-first reads & writes, background sync)
```

7. Modules & Features

7.1 Learn Chapters

Plain-language chapters — **What is AI?**, **Types of AI**, **Study Tools** and **Using AI Responsibly** — introduce core ideas with analogies and illustrations rather than jargon.

7.2 Interactive History of AI

The History section turns 1950–2026 into an explorable timeline: a **year scrubber**, a five-era ribbon (Foundations, the AI Winters, Machine Learning, Deep Learning, the Generative Era), a **focus card** that contrasts "Then" and "Now" for landmark years, quick-jump landmark chips, and a **"Guess the Year"** mini-quiz.

7.3 Visual Learning Lab

The Lab is the practice heart of the site, with two modes:

Mode	Audience	Options
Curriculum Lab	School students	CBSE / ICSE / UK / US / IB · age groups · Math, Science, English, History, Geography
Exam Prep	Competitive-exam aspirants	CUET, Railways, Bank PO, SSC, UPSC · difficulty levels · optional session timer

Both modes present one question at a time with the Eulid tutor, offer a hint before answering and an explanation afterward, track a live streak, and record every answered question to the learner's Firestore progress.

7.4 Find My AI Tool

A recommender that scores a catalogue of 20+ AI tools (from `data/tools-data.js`) against the learner's described need, with subject and "free only" filters.

7.5 Help Centre

A searchable FAQ that opens on a calm grid of topic cards; picking a topic reveals just that topic's questions, and a live search filters across everything.

7.6 Accounts & the Student Dashboard

Learners sign up with email or Google (or browse as a guest). The dashboard shows a profile-picture upload, headline stats (Overall Accuracy, Best Streak, Active Days), a Progress Report chart with Last 7 Days / Last Month / Lifetime ranges, a per-subject breakdown, an activity heatmap and a recent-sessions log.

7.7 Admin Panel

Administrators see every learner in one table — questions answered, accuracy, best streak and last-active — and can open a per-user progress viewer, manage roles, and remove accounts. Access is protected by Firestore rules and a role check.

8. Database Design

Firestore stores two collections. User profiles hold identity and running aggregates; progress is aggregated per day, per mode, per subject so the dashboard and admin views read cheaply.

users/{uid}

```
{ name, email, role: 'student'|'admin',
  curriculum, ageGroup, photo,
  totalQuestions, correctTotal, bestStreak,
  createdAt, lastActive }
```

progress/{uid}/daily/{YYYY-MM-DD__mode__subject}

```
{ date, mode: 'curriculum'|'exam', subject, label,
  questions,   // incremented per answer
  correct,     // incremented when correct
  lastTs }
```

Every answer updates the matching daily document and the user aggregate using atomic `FieldValue.increment` writes with `merge:true`, which are safe to repeat and work offline. Security rules let a learner read and write only their own data, while an administrator (whose own profile carries `role == 'admin'`) may read all profiles and progress and manage roles.

> **Note.** GitHub Pages serves only static files, so the security rules in `firestore.rules` must be published from the Firebase console; otherwise the admin user list cannot load.

9. The Question Engine

`js/questions-engine.js` supplies every Lab and Exam question and can run in two modes controlled by one switch:

- **Local (`'json'`)** — algorithmic math plus a curated JSON bank (`data/question-bank.json`). Free, offline, no cost. This is the default.
- **AI (`'api'`)** — live questions from Claude (`claude-sonnet-4-6`) via a Cloudflare Worker proxy, with the JSON bank as an automatic fallback.

Both paths return the same object shape, so the Lab renderer is unaware of the source.

10. Design System & UX

The interface uses a warm "Sienna" palette (terracotta and earthy browns), Nunito for friendly headings and Inter for readable body text. The brand centres on **Eulid**, a friendly alien mascot whose eyes follow the cursor, and the "Seek-*O*-Sphere" wordmark, whose **O** is a hand-built solar-system SVG. Small touches — an animated neural-network canvas on the home page and a seamless infinite brand marquee — give the site personality without slowing it down.

Token	Value	Role
--brand	#8B3620	Primary sienna
--brand-dark	#46200F	Nav / dark backgrounds
--accent	#C0562F	Terracotta call-to-action
--green	#5F7355	Correct answers / success
--text	#3E2B1E	Body text

11. Testing & Verification

Because the app is static and framework-free, verification focused on real browser behaviour. Pages were rendered headlessly (Chromium) with Firebase mocked to confirm navigation, panel switching and error-free script execution across screen widths. Specific checks included: correct routing of admins to the admin panel without a flash of the student dashboard; the progress-writing path incrementing the right Firestore documents; the dashboard charts reading the daily aggregates correctly; and the responsive brand marquee looping seamlessly from laptop widths up to ultra-wide displays. Firestore security rules were reviewed to ensure learners can access only their own data while admins can read all.

12. Deployment

The site deploys to GitHub Pages from the `main` branch; development happens on `dev` and is merged to `main` to publish. No build step is required — the repository *is* the deployable artifact. Firebase configuration is public web config (safe in client code); data access is governed by Firestore security rules, which are published separately in the Firebase console.

13. Limitations

- Firestore security rules are not deployed by GitHub Pages and must be published manually in the console.
- The contact form collects messages in the UI but has no server-side email delivery.
- Guest (anonymous) sessions do not persist progress by design.
- The AI question mode depends on an external Cloudflare Worker proxy; the default local mode avoids this.

14. Future Scope

- Wire the contact form to an email/serverless backend.
- Add spaced-repetition and personalised revision based on per-subject accuracy.
- Expand the local question bank and add more curricula and languages.
- Introduce achievement badges and class/group leaderboards for teachers.
- Progressive Web App packaging for installable, fully offline use.

15. Conclusion

Seek-O-Sphere demonstrates that a genuinely useful, engaging learning platform can be built on the native web platform alone. By pairing a lightweight vanilla front end with Firebase's managed authentication and database, the project delivers real features — adaptive practice, progress tracking, and administration — while remaining fast, free to host, and easy to maintain. Its clear architecture and friendly design make it both a practical learning tool for its audience and a clean reference implementation of a modern, dependency-free web application.

Seek-O-Sphere · Project Report · Firebase project `ai-classroom-ad779`